Summary — SNOMED ontology pipeline (diabetes test, MIMIC-IV Demo)

Oct 12, 2026 · @youssef

The pipeline works end to end. Adding the SNOMED ontology (model B) improves diabetes prediction over the classical model (model A) in most cases, especially with gradient boosting. But with only 100 patients, these gains remain trends, not proven results.

What we tested

The question: does a model identify diabetic patients better when it is also given the knowledge of a medical ontology?

The data: MIMIC-IV Demo, 100 intensive-care patients, 35 of them diabetic. The model only sees each patient's **other** diseases (hypertension, kidney failure…), plus age and sex. All diabetes diagnoses are removed; otherwise it would simply read the answer.

The ontology: SNOMED CT, downloaded from Athena. It is the "family tree" of diseases: for example, myocardial infarction → ischemic heart disease → cardiovascular disease.

The two models compared:

- **Model A (classical):** one column per disease, 1 if the patient has it, 0 otherwise. The model does not know that two diseases are related.
- **Model B (with ontology):** the knowledge of the tree is added, in three different ways:
  - **Roll-up:** when a patient has a disease, all of its "ancestors" in the tree are ticked too.
  - **node2vec:** each disease becomes a GPS-like coordinate; diseases close in the tree get close coordinates.
  - **KEEP:** the node2vec coordinates, adjusted with the diseases that often occur together in patients.
- **Random control:** randomly drawn coordinates. If it performs as well as model B, the ontology adds nothing.

Each version is tested with three algorithms (gradient boosting, Random Forest, logistic regression), over 25 train/test splits of the patients.

Results: model A vs model B

The score is the **AUPRC**: the higher it is, the better the model finds diabetic patients without false alarms. A model answering at random scores **0.35**. In brackets: the gain over model A. In bold: the best score in each column.

Version	What it adds	Gradient boosting	Random Forest	Logistic regression
Model A	Nothing (reference)	0.539	0.616	0.593
Model B — roll-up	The ancestors of each disease	**0.709 (+0.17)**	0.628 (+0.01)	0.572 (−0.02)
Model B — node2vec (vector)	The tree coordinates	0.650 (+0.11)	0.664 (+0.05)	0.598 (+0.01)
Model B — node2vec (2 distances)	The distance to the “average diabetic”	0.627 (+0.09)	0.630 (+0.01)	0.603 (+0.01)
Model B — KEEP (vector)	Tree + data coordinates	0.628 (+0.09)	**0.700 (+0.08)**	**0.633 (+0.04)**
Model B — KEEP (2 distances)	The distance to the “average diabetic”	0.573 (+0.03)	0.623 (+0.01)	0.600 (+0.01)
Random control	Random coordinates	0.536 (0.00)	0.583 (−0.03)	0.591 (0.00)

Averages over 25 train/test splits. The random control gains nothing: when model B gains, it is thanks to the ontology, not because columns were added.

How to read these results

What holds:

- **Roll-up gives the largest gain:** +0.17 with gradient boosting, better than model A on 23 of the 25 splits. It is also the simplest method.
- **KEEP (vector) is the most consistent version:** the only one that beats model A with all three algorithms.
- **KEEP beats node2vec alone** with 2 algorithms out of 3: adding patient co-occurrences to the ontology helps.
- **The “vector” always beats the “2 distances”:** summarising a patient in two numbers loses information.

- **With the real SNOMED ontology, the gains are clearer** than with the simple ICD code tree tested earlier.

What remains to be proven:

- **No gain is statistically significant.** The best one (roll-up + gradient boosting) has a p-value of 0.11 after correction for multiple comparisons (threshold: 0.05).
- **100 patients is too few:** each test group holds only 7 diabetic patients, so a single misclassified patient shifts the score.
- **Roll-up does not work with logistic regression:** it creates 875 columns for 80 patients, and this model learns noise.

These results validate the pipeline, not the method: the real answer will come from the full MIMIC-IV, with the pipeline frozen as it is.

Three pitfalls found in the data

These three problems distorted the results without raising any error. We found them by comparing two independent methods (ICD codes and the SNOMED hierarchy).

Pitfall	What was happening	Effect	Fix in the pipeline
Different vocabulary versions	The demo was converted with an older SNOMED: the most common diabetes codes (25000, E119) pointed to concepts that have since become “non-standard”	22 of the 35 diabetic patients were counted as non-diabetic	Replace each non-standard concept with its successor (Athena’s “Maps to” relationship)
One code, several concepts	A single ICD code can yield several SNOMED concepts, e.g. “type 2 diabetes” + “kidney disease due to diabetes”	The second one stayed in the features: the model was reading the answer (AUC of 0.94, too good to be true)	Remove every row coming from a diabetes source code

Pitfall	What was happening	Effect	Fix in the pipeline
Complications stored elsewhere	In SNOMED, diabetes complications are not under “Diabetes mellitus” but under “Complication due to diabetes mellitus”	The target was incomplete	Define the target with several roots + a keyword check

Key takeaway for the thesis: **the vocabulary version used to convert the data must match the ontology’s version**, and the target must be checked with an independent method.